

Содержит А

В Python вы можете проверить, содержит ли строка подстроку, используя ключевое слово `in`. Например, строка:

```
return "I" in "Team"
```

вернет `False`, так как в "Team" нет "I". Тем не менее:

```
return "I" in "I love Python"
```

возвращает `True`, потому что в слове «I love Python» есть «I».

Помните, что для создания лямбда-функции вы можете использовать синтаксис:

```
lambda my_input: <my_input modified somehow>
```

Например, лямбда, которая вернет `my_input` плюс 1, будет выглядеть так:

```
plus_one = lambda my_input: my_input+1
```

Задание

1. Создайте лямбда-функцию с именем `contains_a`, которая принимает входное слово и возвращает `True`, если вход содержит букву 'a'. В противном случае верните `False`.

Длинная строка

Чтобы найти количество символов в строке, мы используем `len`. Этот блок кода:

```
print(len("Hello"))  
print(len("world!"))  
print(len("Hello, world!"))
```

Выведите:

```
5  
6  
13
```

Задание

1. Создайте лямбда-функцию с именем `long_string`, которая принимает входную строку и возвращает `True`, если строка содержит более 12 символов. В противном случае верните `False`.

Заканчивается на А

Вы можете получить символ строки `имя_строки`, используя синтаксис `string_name [индекс]`, где индекс — это место символа, который вы хотите получить, начиная с 0. Последний символ в строке - `имя_строки [-1]`.

Например:

```
my_string = "Whoa! A seesaw"
print(my_string[0])
print(my_string[2])
print(my_string[-1])
```

Выведет

```
"W"
"o"
"w"
```

Задание

1. Создайте лямбда-функцию с именем `end_in_a`, которая принимает входную строку и возвращает `True`, если последний символ в строке - `a`. В противном случае верните `False`.

Удвоение или ноль

Python упрощает математику. Для умножения вы можете использовать `*`:

```
>>> 4*2
8
>>> 2*3
6
>>> 0*10
0
>>> 20*10
200
```

Напоминаем, что для возврата разных выходных данных в зависимости от разных входных данных мы можем использовать if и else внутри нашей лямбда-функции:

```
add_or_subtract = lambda input_number: input_number - 1 if
input_number >= 0 else input_number + 1
```

Функция add_or_subtract вернет ваш ввод минус 1, если ваш ввод положительный или 0, в противном случае вернет ваш ввод плюс 1.

Вот несколько примеров того, как это будет работать:

```
>>> add_or_subtract(0)
-1
>>> add_or_subtract(8)
7
>>> add_or_subtract(-4)
-3
```

Четное и нечетное

В Python% или оператор по модулю возвращает остаток после деления.

```
>>> 4%2 #This divides evenly
0
>>> 7%3 #7/3 has a remainder of 1
1
>>> 27%10
7
>>> 30%10
0
```

Y

Вы можете использовать% 2, чтобы определить, четное или нечетное число. Если он четный, остатка быть не должно (выход 0). Если он нечетный, должен быть остаток 1:

```
>>> 4%2
0
>>> 7%2
1
>>> 9%2
1
>>> 0%2
0
```

Задание

1. Создайте лямбда-функцию с именем `even_or_odd`, которая принимает целое число с именем `num`. Если число четное, верните «четное». Если число нечетное, верните «нечетное».

Кратно трем

В общем, использование `%` `n` скажет вам, кратно ли целое число `n`. Если результат равен 0, целое число кратно `n` (так как в делении нет остатка):

```
>>> 4%4 #4 is a multiple of 4
0
>>> 12%5 #12 is not a multiple of 5
2
>>> 9%2 #9 is not a multiple of 2
1
>>> 100%10 #100 is a multiple of 10
0
```

Задание

1. Создайте лямбда-функцию с именем `multiple_of_three`, которая принимает целое число с именем `num`. Если число кратно трем, верните «кратное трем». В противном случае верните «не кратное».

Рейтинг фильмов

Сравнения можно проводить с помощью:

`<`: меньше чем

`<=`: меньше или равно

`>`: больше чем

`>=`: больше или равно

`==`: равно

`!=`: Не равно

Эти утверждения возвращают либо `True`, либо `False`:

```
>>> 4 < 4
False
>>> 4 <= 4
True
>>> 9 > 2
True
```

```
>>> 9 >= 2
True
>>> 1 > 2
False
```

Задание

Создайте лямбда-функцию с именем `rate_movie`, которая принимает число с именем `rating`. Если рейтинг больше 8,5, верните «Мне понравился этот фильм». В противном случае верните «Этот фильм был не очень хорошим».